

Roadmap Completo de Arquitetura e Desenvolvimento Fullstack

Plataforma de Eventos e Ingressos

Frontend	Next.js + React + TypeScript
Backend	Python + FastAPI
Banco	PostgreSQL + SQLAlchemy + Alembic
Prazo do desafio	7 dias corridos

Documento de planejamento técnico baseado no PDF oficial do Desafio Elite Dev 2026.

Objetivo: entregar um fluxo completo, coerente, testável e fácil de defender tecnicamente.

Sumário

1. Leitura estratégica do desafio
2. Stack recomendada
3. Arquitetura geral
4. Estrutura do repositório
5. Arquitetura do frontend
6. Arquitetura do backend FastAPI
7. Modelagem do banco de dados
8. Autenticação e autorização
9. Integração com Ticketmaster
10. Fluxo de reserva e concorrência
11. Pagamento simulado
12. Emissão de ingresso e QR seguro
13. Compartilhamento de ingresso
14. Portaria e validação
15. Contrato da API
16. Tratamento de erros
17. Testes
18. Docker e ambientes
19. Deploy
20. Roadmap de 7 dias
21. Estratégia de commits
22. README e uso de IA
23. Checklist de entrega
24. Como defender a arquitetura na entrevista

1. Leitura estratégica do desafio

O produto pedido é uma plataforma de eventos e ingressos em que o organizador publica eventos a partir de um catálogo externo, o cliente reserva e paga de forma simulada, recebe um ingresso com QR e a portaria valida a entrada. O escopo oficial também exige armazenamento de eventos, reservas e ingressos, autenticação com três papéis e proteção contra venda ou validação duplicada.

Princípio central O PDF deixa claro que o objetivo não é volume de funcionalidades. A avaliação privilegia decisões, organização, fluxo completo, documentação e justificativas técnicas. Portanto, a estratégia deve ser: primeiro fechar o fluxo de ponta a ponta; depois adicionar diferenciais.

MVP que deve funcionar sem falhas

1. Login com Organizador, Cliente e Portaria.
2. Organizador pesquisa um item no catálogo externo e publica um evento local.
3. Cliente navega pelos eventos e cria uma reserva por quantidade.
4. Pagamento simulado retorna aprovado ou recusado.
5. Pagamento aprovado gera um ou mais ingressos individuais.
6. Cliente visualiza QR e pode compartilhar o ingresso por link.
7. Portaria lê QR pela câmera ou aceita código manual.
8. Backend retorna: válido, inválido, já utilizado ou evento errado.

Escopo que eu deixaria para depois

- Mapa de assentos em tempo real.
- Integração com provedor financeiro real.
- Recuperação de senha e envio por e-mail.
- Microserviços, filas e cache distribuído.
- Aplicativo nativo.

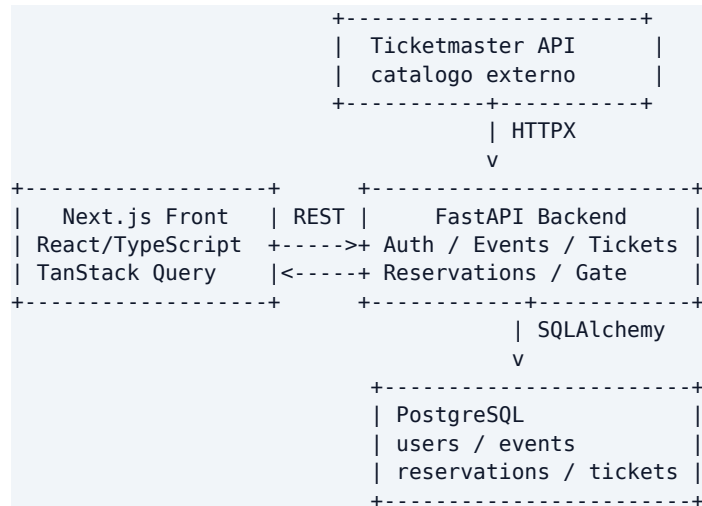
2. Stack recomendada

Camada	Tecnologia	Motivo
Frontend	Next.js + React + TypeScript	React é obrigatório e Next.js agiliza rotas, deploy e organização.
UI	Tailwind CSS + shadcn/ui	Rapidez com controle visual suficiente para evitar aparência genérica.
Dados no front	TanStack Query	Cache, estados de loading/error e invalidação previsível.
Backend	Python + FastAPI	API enxuta, tipagem, documentação OpenAPI e bom encaixe com o desafio.
ORM	SQLAlchemy 2	Controle explícito de transações e bom suporte a PostgreSQL.
Migrations	Alembic	Schema versionado e reproduzível.
Banco	PostgreSQL	Consistência, constraints, transações e locking.
Validação	Pydantic	Schemas de entrada e saída claros.

HTTP externo	HTTPX	Cliente async para Ticketmaster.
Auth	JWT + Argon2/bcrypt	Autenticação simples e adequada ao escopo.
QR	PyJWT + qrcode	Token assinado e QR não trivialmente forjável.
Testes	Pytest + HTTPX	Testes unitários e de integração da API.
Infra	Docker Compose	Ambiente local previsível.

Escolha de arquitetura Monólito modular. O prazo é curto e o domínio é pequeno. A modularização preserva organização sem custo operacional de microserviços.

3. Arquitetura geral



O frontend nunca acessa o banco nem a Ticketmaster diretamente. O backend é o ponto único de validação de regras de negócio, autorização, integração externa e persistência.

4. Estrutura do repositório

```

elite-events/
|-- frontend/
|   |-- Next.js + TypeScript
|-- backend/
|   |-- FastAPI + SQLAlchemy
|-- docs/
|   |-- architecture.md
|   |-- decisions.md
|   |-- ai-usage.md
|-- docker-compose.yml
|-- README.md
`-- .gitignore

```

A separação frontend/backend facilita deploy independente e deixa claro para o avaliador onde cada responsabilidade está. O diretório docs versiona artefatos de raciocínio, algo explicitamente valorizado no desafio.

5. Arquitetura do frontend

```
frontend/src/
|-- app/
|   |-- (public)/
|   |   |-- page.tsx
|   |   |-- events/[id]/page.tsx
|   |-- (auth)/login/
|   |-- customer/
|   |   |-- checkout/
|   |   |-- tickets/
|   |-- organizer/
|   |   |-- dashboard/
|   |   |-- events/
|   |-- gate/page.tsx
|-- components/
|   |-- ui/
|   |-- events/
|   |-- checkout/
|   |-- tickets/
|   |-- gate/
|-- services/
|-- hooks/
|-- schemas/
|-- types/
|-- utils/
```

Responsabilidades

- **app/** rotas e composição de páginas.
- **components/** componentes de interface e elementos reutilizáveis.
- **services/** comunicação HTTP com a API FastAPI.
- **hooks/** regras de consumo de dados e interação.
- **schemas/** validações Zod dos formulários.
- **types/** contratos TypeScript retornados pela API.

Rotas principais

Papel	Rotas
Público	/, /events, /events/[id], /login
Cliente	/checkout/[eventId], /my-tickets, /my-tickets/[id]
Organizador	/organizer/dashboard, /organizer/events, /organizer/events/new
Portaria	/gate

UX Evite hero genérico, excesso de gradientes e cards artificiais. Priorize navegação simples, dados do evento, estados vazios, feedback de erro e fluxo de compra evidente.

6. Arquitetura do backend FastAPI

```
backend/
|-- app/
|   |-- main.py
```

```

|-- core/
|   |-- config.py
|   |-- security.py
|   |-- exceptions.py
|   |-- logging.py
|-- database/
|   |-- base.py
|   |-- session.py
|   |-- seed.py
|-- models/
|-- modules/
|   |-- auth/
|   |-- users/
|   |-- catalog/
|   |-- events/
|   |-- reservations/
|   |-- payments/
|   |-- tickets/
|   |-- gate/
|-- integrations/ticketmaster/
|-- shared/
|-- tests/
-- migrations/
-- alembic.ini
-- requirements.txt
-- Dockerfile
-- .env.example

```

Padrão interno dos módulos

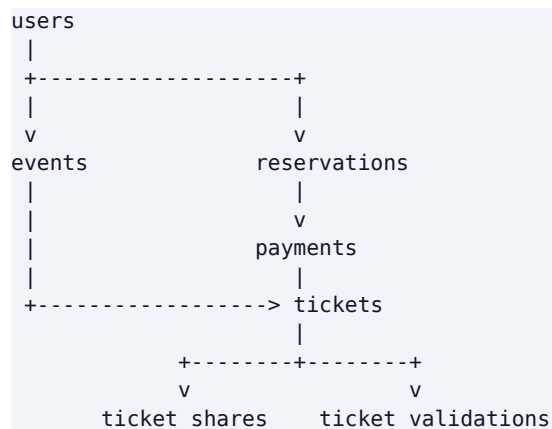
```

modules/events/
|-- router.py      # HTTP, status codes e dependencies
|-- schemas.py     # Pydantic request/response
|-- service.py     # regras de negocio
-- repository.py   # persistencia SQLAlchemy

```

O caminho de uma requisição fica previsível: Router -> Service -> Repository -> PostgreSQL. Regras críticas, como estoque, pagamento e validação de ingresso, permanecem no service e são protegidas por transações.

7. Modelagem do banco de dados



users

Campo	Tipo / regra
-------	--------------

id	UUID PK
name	varchar
email	varchar UNIQUE
password_hash	varchar
role	ORGANIZER CUSTOMER GATE
created_at	timestamp
updated_at	timestamp

events

Campo	Tipo / regra
id	UUID PK
organizer_id	UUID FK users
external_provider	ticketmaster
external_id	id externo
title	varchar
description	text
image_url	text
venue_name	varchar
venue_address	text
event_date	timestamp
capacity	integer > 0
available_tickets	integer >= 0
ticket_price	numeric(10,2)
status	DRAFT PUBLISHED CANCELLED

reservations, payments e tickets

Tabela	Campos essenciais
reservations	id, customer_id, event_id, quantity, unit_price, total_amount, status, expires_at
payments	id, reservation_id UNIQUE, amount, status, provider, failure_reason
tickets	id, reservation_id, event_id, owner_id, public_code UNIQUE, qr_token_hash, status, used_at
ticket_shares	id, ticket_id, token_hash, expires_at, revoked_at
ticket_validations	id, ticket_id, gate_user_id, event_id, result, validated_at

Decisão importante Se o cliente compra 3 ingressos, gere 3 tickets individuais. Isso simplifica compartilhamento, QR, auditoria e validação unitária.

8. Autenticação e autorização

O login valida e-mail e senha, gera um JWT e inclui no payload apenas o necessário para identificação e autorização. Senhas ficam sempre com hash forte; tokens e segredos permanecem em variáveis de ambiente.

```
POST /api/v1/auth/login
  |
  v
AuthService
  |
  +-----+
  | validar email/senha |
  | gerar JWT           |
  +-----+

JWT payload:
{
  "sub": "user-uuid",
  "role": "CUSTOMER"
}
```

RBAC por papel

Papel	Permissões principais
ORGANIZER	criar, editar, publicar e gerenciar próprios eventos
CUSTOMER	reservar, pagar, visualizar e compartilhar ingressos
GATE	validar ingresso na portaria

No FastAPI, dependencies como `get_current_user` e `require_roles` centralizam autenticação e autorização, evitando repetição nas rotas.

9. Integração com Ticketmaster

```
Next.js
  |
  v
GET /api/v1/catalog/events?q=coldplay
  |
  v
FastAPI CatalogService
  |
  v
TicketmasterClient (HTTPX)
  |
  v
Ticketmaster API
```

O backend deve mapear a resposta externa para um DTO interno simples. Quando o organizador escolhe um item e publica o evento, copie para o PostgreSQL os dados necessários, como título, imagem, descrição e identificador externo. A partir daí, a tela pública do evento não depende da Ticketmaster.

Segurança A API key da Ticketmaster fica apenas no backend. O frontend nunca recebe a credencial.

10. Fluxo de reserva e concorrência

O desafio exige impedir venda além da disponibilidade. A validação no frontend é útil para UX, mas não garante integridade. A decisão correta deve ocorrer no banco, dentro de uma transação.

```
Cliente seleciona 2 ingressos
  |
  v
POST /events/{id}/reservations
  |
  v
BEGIN TRANSACTION
  |
  v
SELECT event FOR UPDATE
  |
  v
available_tickets >= quantity ?
  |           |
  nao         sim
  |           |
  erro 409    diminuir estoque
              |
              v
            criar reserva
              |
              v
            COMMIT
```

Por que isso importa

Se existir apenas 1 ingresso e dois clientes tentarem reservar ao mesmo tempo, um lock pessimista ou um update condicional garante que somente uma transação finalize. Essa é uma das decisões de backend mais fortes para explicar na entrevista.

Reserva temporária

Como melhoria opcional, use expires_at (por exemplo, 10 minutos). Reserva paga vira PAID; reserva expirada devolve quantidade ao estoque. Não implemente isso antes do MVP completo.

11. Pagamento simulado

```
PaymentService
  |
  v
PaymentGateway
  |
  v
FakePaymentGateway

Regra de teste:
cartao terminando em 0000 -> DECLINED
outro numero de teste     -> APPROVED
```

Separar o gateway do serviço permite substituir a simulação por Stripe ou outro provedor no futuro sem reescrever o domínio. O pagamento recusado não deve gerar ingresso; o aprovado deve alterar a reserva e criar os tickets.

12. Emissão de ingresso e QR seguro

Cada ticket tem um `public_code` legível para digitação manual e um token assinado para o QR. O QR não deve conter apenas um ID previsível.

```
public_code: ELT-8D72-A93C
```

```
QR payload assinado:
```

```
{
  "ticket_id": "uuid",
  "event_id": "uuid",
  "type": "ticket"
}
```

```
assinatura: HMAC/JWT com TICKET_SECRET
```

O QR pode ser gerado sob demanda a partir do token. Se optar por armazenar apenas o hash do token, você reduz impacto de eventual leitura direta do banco.

13. Compartilhamento de ingresso

O compartilhamento deve usar um token aleatório, não o ID do ticket. A aplicação gera um link público limitado ao ingresso compartilhado.

```
POST /tickets/{ticket_id}/share
```

```
|
```

```
v
```

```
gerar token aleatorio
```

```
|
```

```
v
```

```
salvar token_hash + expires_at
```

```
|
```

```
v
```

```
https://app.exemplo.com/ticket/shared/<token>
```

Opcionalmente, permita revogação e expiração. O endpoint público nunca deve permitir alteração de dados do ticket.

14. Portaria e validação

A tela da portaria deve priorizar velocidade: câmera primeiro, código manual como alternativa e feedback visual evidente. O backend precisa distinguir quatro resultados: `VALID`, `INVALID`, `ALREADY_USED` e `WRONG_EVENT`.

```
QR / codigo manual
```

```
|
```

```
v
```

```
POST /api/v1/gate/validate
```

```
|
```

```
v
```

```
verificar assinatura / localizar ticket
```

```
|
```

```
+--> nao existe -----> INVALID
```

```
|
```

```
+--> evento diferente --> WRONG_EVENT
```

```

|
+--> status USED -----> ALREADY_USED
|
v
BEGIN + SELECT FOR UPDATE
|
v
status = USED / used_at = now()
|
v
registrar ticket_validation
|
v
COMMIT -> VALID

```

Concorrência novamente A validação também precisa de transação. Duas catracas lendo o mesmo QR quase ao mesmo tempo não podem liberar duas entradas.

15. Contrato da API

Módulo	Endpoints principais
Auth	POST /auth/register POST /auth/login GET /auth/me
Catalog	GET /catalog/events?q=
Events	GET /events GET /events/{id} POST /events PATCH /events/{id} DELETE /events/{id}
Organizer	GET /organizer/events GET /organizer/events/{id}/stats
Reservations	POST /events/{id}/reservations GET /reservations/{id} POST /reservations/{id}/cancel
Payments	POST /reservations/{id}/payments
Tickets	GET /me/tickets GET /tickets/{id} POST /tickets/{id}/share
Shared	GET /shared-tickets/{token}
Gate	POST /gate/validate

Use prefixo /api/v1 para deixar o contrato explícito. O FastAPI documentará o conjunto automaticamente em /docs e /openapi.json.

16. Tratamento de erros

Padronize respostas para que o frontend não precise interpretar mensagens livres.

```

{
  "error": {
    "code": "INSUFFICIENT_TICKETS",
    "message": "Nao existem ingressos suficientes disponiveis."
  }
}

```

- INVALID_CREDENTIALS
- FORBIDDEN

- EVENT_NOT_FOUND
- EVENT_SOLD_OUT
- RESERVATION_EXPIRED
- PAYMENT_DECLINED
- INVALID_TICKET
- TICKET_ALREADY_USED
- WRONG_EVENT

Mapeie cada erro para status HTTP coerente, por exemplo 400/401/403/404/409/422 conforme o caso.

17. Testes

Poucos testes bem escolhidos mostram mais maturidade que cobertura artificial. Priorize regras de negócio e concorrência.

Área	Cenário
Auth	organizer cria evento; customer recebe 403
Reserva	não permite quantidade acima da disponibilidade
Concorrência	duas reservas simultâneas não ultrapassam capacidade
Pagamento	recusado não gera ticket
Pagamento	aprovado gera exatamente N tickets
QR	token adulterado é inválido
Portaria	ticket válido muda para USED
Portaria	ticket usado retorna ALREADY_USED
Portaria	ticket de outro evento retorna WRONG_EVENT

Ferramentas: Pytest, pytest-asyncio, HTTPX AsyncClient e banco de teste isolado. Para frontend, Vitest/React Testing Library somente nos fluxos que realmente tragam valor.

18. Docker e ambientes

No primeiro momento, use Docker Compose para o PostgreSQL e rode frontend/backend localmente. Depois, se houver tempo, containerize tudo.

```
docker-compose.yml

services:
  db:
    image: postgres
    environment:
      POSTGRES_DB: elite
      POSTGRES_USER: elite
      POSTGRES_PASSWORD: elite
    ports:
      - "5432:5432"
```

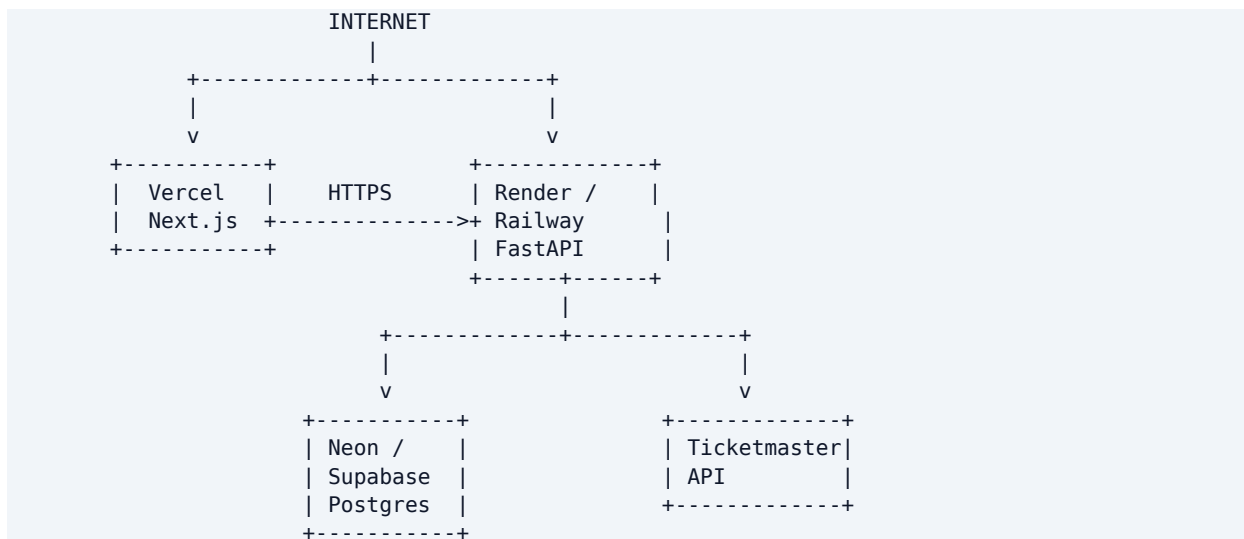
Variáveis de ambiente

```
DATABASE_URL=postgresql+asyncpg://...
JWT_SECRET=...
JWT_ALGORITHM=HS256
```

```
ACCESS_TOKEN_EXPIRE_MINUTES=60
TICKET_SECRET=...
TICKETMASTER_API_KEY=...
FRONTEND_URL=http://localhost:3000
```

Versione .env.example, nunca o .env real.

19. Deploy



O deploy não é obrigatório no enunciado, mas facilita a avaliação e rende ponto adicional. Por isso, trate-o como meta de entrega, não como luxo.

20. Roadmap de 7 dias

Dia	Objetivo	Entregável
1	Fundação	Repo, Next.js, FastAPI, PostgreSQL, SQLAlchemy, Alembic, models e seed.
2	Auth + Eventos	JWT, roles, Ticketmaster, CRUD e publicação de evento.
3	Reserva	Fluxo por quantidade, transaction e proteção contra overselling.
4	Pagamento + Ticket	Fake gateway, aprovado/recusado, geração de tickets e QR.
5	Portaria + Share	Scanner, código manual, validação transacional e link compartilhável.
6	Qualidade	Testes críticos, tratamento de erros, loading/empty states, responsividade.
7	Entrega	Deploy, README, screenshots, revisão, bugs e documentação de decisões.

Regra de priorização

Não abrir nova frente antes do fluxo fechar Se no Dia 4 ainda não existe compra -> pagamento -> ingresso, não implemente mapa de assentos, dashboards sofisticados ou animações. Termine o caminho principal primeiro.

21. Estratégia de commits

```
chore: initialize frontend and backend
feat(auth): implement jwt authentication and role guards
feat(events): add ticketmaster catalog integration
feat(events): allow organizers to publish events
feat(reservations): add atomic ticket reservation
feat(payments): implement simulated payment flow
feat(tickets): generate signed qr tickets
feat(gate): add ticket validation flow
test(tickets): prevent duplicate ticket validation
docs: document architecture and technical decisions
```

Evite concentrar tudo em um único commit final. O histórico é parte do processo de avaliação.

22. README e uso de IA

O README deve permitir que alguém rode e entenda o projeto sem falar com você. Inclua instruções de banco, migrations, seed, usuários de teste, variáveis de ambiente e limitações conhecidas.

```
README.md
|-- Visao geral
|-- Demo / URLs
|-- Arquitetura
|-- Stack
|-- Decisoões técnicas
|-- Como executar
|-- Variáveis de ambiente
|-- Migrations e seed
|-- Usuarios de teste
|-- Testes
|-- Uso de IA
|-- Trade-offs
`-- Melhorias futuras
```

Seção de uso de IA

O próprio desafio recomenda transparência. Registre ferramentas usadas, onde ajudaram e quais decisões foram suas. Versione docs/ai-usage.md e docs/decisions.md.

Exemplo:
Ferramentas: ChatGPT e GitHub Copilot.
Usei IA para revisar modelagem, discutir concorrência, gerar casos de teste e revisar documentação.
Decisões manuais: arquitetura, prioridades do MVP, UX, modelagem final, regras de reserva e validação.

23. Checklist de entrega

☐ Repositório público no GitHub.

- ☐ Commits distribuídos ao longo do desenvolvimento.
- ☐ README reproduzível.
- ☐ Organizador semeado.
- ☐ Dois clientes semeados.
- ☐ Usuário de portaria semeado.
- ☐ Pelo menos um evento publicado com ingressos disponíveis.
- ☐ Busca/navegação de eventos funcionando.
- ☐ Reserva funcionando.
- ☐ Pagamento aprovado e recusado funcionando.
- ☐ Ingresso e QR funcionando.
- ☐ Compartilhamento por link funcionando.
- ☐ Portaria via câmera funcionando.
- ☐ Digitação manual funcionando.
- ☐ VALID, INVALID, ALREADY_USED e WRONG_EVENT cobertos.
- ☐ Proteção contra overselling implementada no backend.
- ☐ Proteção contra dupla validação implementada no backend.
- ☐ Testes críticos passando.
- ☐ Aplicação publicada.
- ☐ Limitações conhecidas documentadas.
- ☐ Uso de IA documentado.

24. Como defender a arquitetura na entrevista

Por que FastAPI?

Porque o sistema é orientado a API, Python é permitido no desafio e o FastAPI oferece tipagem, validação com Pydantic e documentação OpenAPI automática sem impor peso excessivo.

Por que PostgreSQL?

Porque reserva e validação exigem consistência. Transações, constraints e locking permitem impedir overselling e dupla utilização do ticket.

Por que monólito modular?

Porque o prazo é de 7 dias e o domínio ainda é pequeno. Separar por módulos mantém clareza sem adicionar complexidade operacional de microserviços.

Por que começar por ingresso de pista/quantidade?

Porque o enunciado permite escolher entre mapa de assentos e quantidade. A opção por quantidade reduz complexidade e permite fechar o fluxo completo antes de atacar opcionais.

Qual foi a decisão técnica mais importante?

Garantir consistência no banco: tanto a reserva quanto a validação do ingresso são transacionais. Isso impede que duas requisições concorrentes vendam o último ingresso ou validem o mesmo QR duas vezes.

O que eu faria depois do MVP?

- Reserva temporária com expiração e devolução automática ao estoque.
- Mapa de assentos.

- Dashboard de organizador com métricas.
- Busca e filtros refinados.
- Rate limiting e observabilidade melhor.
- CI com testes automáticos no GitHub Actions.

Mensagem final O melhor projeto para esse desafio não é o maior. É o que fecha o fluxo, protege as regras críticas, tem decisões conscientes e deixa claro no código e no README por que cada escolha foi feita.

Referência do desafio

Documento base: Desafio Elite Dev 2026 - plataforma de eventos e ingressos. Requisitos utilizados neste roadmap: React no frontend; Node.js, Python ou Java no backend; banco de dados à escolha; três papéis de autenticação; integração com Ticketmaster Discovery ou TMDb; reserva, pagamento simulado, QR, compartilhamento, portaria, seeds, documentação, histórico de commits e incentivo a deploy e testes.